

Full Stack Developer Internship Assignment

Debales AI

1. Assignment Overview

Field	Detail
Title	Multi-tenant AI Assistant (product-style) with admin dashboard driven by MongoDB config
Objective	Show you can model projects + product instances + users, enforce access, use a layered API, integrate AI in a controlled way, and build an admin dashboard whose layout/content is driven by MongoDB
Duration	2–3 days (~16–24 hours).
Skills tested	Next.js App Router, React, TypeScript, MongoDB, Zod, TanStack Query, multi-tenant thinking, server-side authorization, API design, UX (loading/error/empty), real AI API.

2. Requirements

Frontend

- **Next.js (App Router)** + React + TypeScript + Tailwind CSS.
- **Dynamic routes** (e.g. project slug; chat/conversation routes as you design).
- **Product area (chat shell)**: sidebar or layout with conversations, new chat, and navigation to chat—may be implemented with normal React components (hard-coded or conventional routing is fine). Config-driven rendering is not required here.
- **Chat UI**: messages (user + assistant), optional “step” lines (e.g. analyzing, calling integration), loading states.

- **Project admin dashboard (admin-only):** must use config-driven components—structure, sections, labels, widget list/order, or similar loaded from MongoDB so editing the DB changes the dashboard without a code change.
- **TanStack Query** for server state; no direct DB calls from UI components.

Backend

- Next.js Route Handlers.
- **Mandatory layering:** Access (pure rules where possible) → Services (business logic + data access) → Routes (thin) → Hooks (client) → UI.
- **Zod** validation on inputs (query/body).
- **Controlled AI flow:** service layer decides when to call AI; real free-tier API (e.g. Gemini / OpenRouter).
- **Integration simulation:** at least two (e.g. Shopify-style + CRM-style), toggleable per project (or per product instance), stored in MongoDB; chat flow should reflect toggles + mock data.

Database

- MongoDB + Mongoose with types aligned to Zod where practical.
- **Projects** (tenant boundary; e.g. namespace/slug).
- **Product instances:** link a product (e.g. “AI Sales Assistant”) to a project (conceptually like productInstance: projectId, nameSpace, productType, etc.—minimal fields OK).
- **Users** associated to projects with roles (e.g. admin vs member—simplified is fine).
- **Conversations + messages** scoped to a project and product instance.
- **Admin dashboard config (required)**
 - Store a dashboard layout / insights / widgets document in MongoDB (you define the schema).
 - The admin dashboard page must read this config and render components accordingly (e.g. map config entries to card types or sections).
 - Outside the admin dashboard, config-driven components are optional; the rest of the UI does not need to be DB-driven.

Authentication

- Full auth not required. Acceptable: seed users, cookie/session stub, or fixed “login as” for demo.
- **Required:** authorization checks on the server (who can open which project; only admins can open the admin dashboard).

Deployment

- Deploy to Vercel (or similar) or provide Docker + README so we can run locally.
- **README** must list env vars, seed steps, and which collection/document drives the admin dashboard.
- **Proof of config-driven admin UI:** document how to edit that MongoDB document and see only the admin dashboard change and include a short video.

3. Bonus Features (optional)

- Config-driven tabs or nav in the main product shell (not required for core score).
- Unit test for pure access rules or Zod schemas.
- data-testid on main regions.
- Real AI API with clear rate-limit handling and fallbacks.

4. Submission Guidelines

- Git repository (public link or zip).
- **README:** setup, run, seed, env, architecture (1 page: layers + multi-tenant model).
- Assumptions and what is mocked (AI, integrations).
- Clear note: where config-driven behavior lives (admin dashboard only for core requirements).
- 5–10 min Loom (walkthrough + MongoDB edit → admin dashboard UI change).

5. Evaluation Criteria

Skill Area	Points
Multi-tenant model (project, product instance, scoped conversations)	25
Access / authorization (server-enforced; admin route protected)	15
Layered API (Access → Services → Routes → Hooks → UI) + Zod	20
Chat + controlled AI + integration toggles + mock behavior	15
Admin dashboard: config-driven components (MongoDB drives layout/sections/widgets; verifiable)	15
Frontend UX & code quality (states, structure, readability)	10
Total	100

Passing guidance: ~70+. For the admin dashboard item, we expect **visible UI changes when the stored config changes** (demo, video, or step-by-step in README)—not only screenshots of a static page.

for candidates:

Config-driven components are required only on the project admin dashboard; the chat and main product UI may be built with standard components and routing.